AI23531 – DEEP LEARNING

EX 2 MULTI-LAYER PERCEPTRON (MLP) FOR CLASSIFICATION (IRIS DATASET)

AIM

To develop a Multi-Layer Perceptron for a classification task, experiment with different numbers of hidden layers and activation functions, and evaluate performance using accuracy and loss.

PROCEDURE

1. Import the required libraries (NumPy, Matplotlib, scikit-learn, Keras).
2. Load the Iris dataset, one-hot encode the labels and standardise the features.
3. Split the data into 80% training and 20% test sets.
4. Define a function that builds an MLP for a given list of hidden layers and activation.
5. Train nine models: hidden layers [8], [16, 8], [32, 16, 8] with ReLU, tanh and sigmoid activations (50 epochs, batch size 5).
6. Record the test accuracy and loss of each model.
7. Plot the accuracy and loss curves of the last model and predict the species for user-entered measurements.

ALGORITHM

1. Start.
2. Load Iris data; apply one-hot encoding to the labels and StandardScaler to the features.
3. Split into training and test sets (test size 0.2).
4. For each hidden-layer configuration and each activation function: build the MLP (hidden Dense layers + Softmax output), compile with Adam and categorical cross-entropy.
5. Train the model for 50 epochs with 10% validation split.
6. Evaluate on the test set and print accuracy and loss.
7. Plot training/validation accuracy and loss.
8. Read the four flower measurements from the user, scale them, predict the class with the model and print the species.
9. Stop.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

iris = load_iris()
X = iris.data
y = iris.target.reshape(-1, 1)
classes = iris.target_names

encoder = OneHotEncoder(sparse_output=False)
```

```
y_encoded = encoder.fit_transform(y)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42
)

def create_mlp(input_dim, output_dim, hidden_layers, activation='relu'):
    model = Sequential()
    model.add(Dense(hidden_layers[0], input_dim=input_dim, activation=activation))

    for units in hidden_layers[1:]:
        model.add(Dense(units, activation=activation))

    model.add(Dense(output_dim, activation='softmax'))

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

hidden_layer_configs = [[8], [16, 8], [32, 16, 8]]
activations = ['relu', 'tanh', 'sigmoid']

for hidden_layers in hidden_layer_configs:
    for activation in activations:
        print(f"\nTesting MLP with hidden_layers={hidden_layers},
activation={activation}")

        model = create_mlp(
            input_dim=4,
            output_dim=3,
            hidden_layers=hidden_layers,
            activation=activation
        )

        history = model.fit(
            X_train,
            y_train,
            epochs=50,
            batch_size=5,
            verbose=0,
            validation_split=0.1
        )

        test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)

        print(f"Test Accuracy: {test_acc:.4f}")
        print(f"Test Loss: {test_loss:.4f}")

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Model Accuracy')
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Model Loss')
plt.legend()

plt.tight_layout()
plt.show()

sepal_length = float(input("Sepal length (cm): "))
sepal_width = float(input("Sepal width (cm): "))
petal_length = float(input("Petal length (cm): "))
petal_width = float(input("Petal width (cm): "))

user_input = np.array([[
    sepal_length,
    sepal_width,
    petal_length,
    petal_width
]])

user_input_scaled = scaler.transform(user_input)

prediction = model.predict(user_input_scaled, verbose=0)
predicted_class_index = np.argmax(prediction)
predicted_class_name = classes[predicted_class_index]

print("\nPredicted Iris Species:", predicted_class_name)
```

OUTPUT

```
Testing MLP with hidden_layers=[8], activation=relu
Test Accuracy: 0.9000
Test Loss: 0.2973

Testing MLP with hidden_layers=[8], activation=tanh
Test Accuracy: 0.9667
Test Loss: 0.1676

Testing MLP with hidden_layers=[8], activation=sigmoid
Test Accuracy: 0.9333
Test Loss: 0.4032

Testing MLP with hidden_layers=[16, 8], activation=relu
Test Accuracy: 1.0000
Test Loss: 0.0820

Testing MLP with hidden_layers=[16, 8], activation=tanh
Test Accuracy: 0.9667
Test Loss: 0.0887

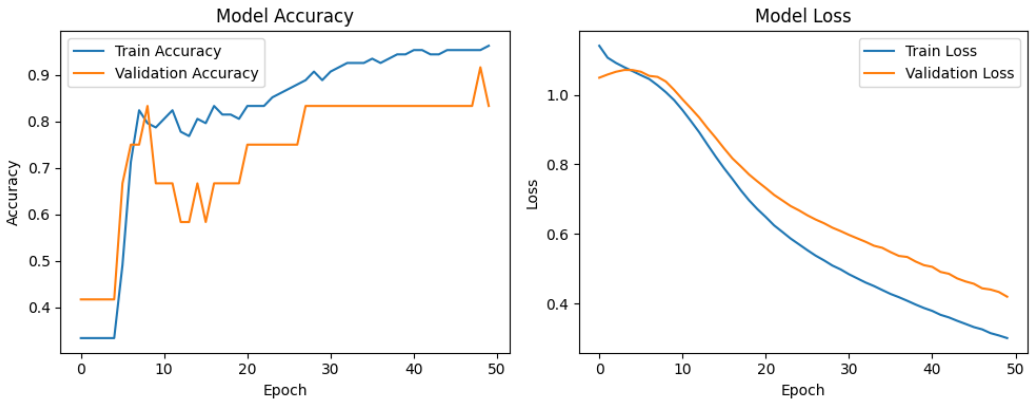
Testing MLP with hidden_layers=[16, 8], activation=sigmoid
Test Accuracy: 0.9667
Test Loss: 0.4523

Testing MLP with hidden_layers=[32, 16, 8], activation=relu
Test Accuracy: 0.9667
Test Loss: 0.0463

Testing MLP with hidden_layers=[32, 16, 8], activation=tanh
Test Accuracy: 0.9667
Test Loss: 0.0559

Testing MLP with hidden_layers=[32, 16, 8], activation=sigmoid
Test Accuracy: 0.9333
Test Loss: 0.3004
```

Console output



Accuracy and loss curves of the MLP

RESULT

The best accuracy (100%) was obtained with hidden layers [16, 8] and ReLU, while the lowest test loss (0.0463) came from [32, 16, 8] with ReLU. Sigmoid gave the highest losses overall.